

Breadth-First Search (BFS)

Algorithm Design → Search (Series: 8 parts)

Discrete Maths for Engineers

Leandro Rodrigues

March 4, 2026

Algorithm Design → Search

Leandro Rodrigues

Contents

1	Graph Model & Problem Setting	3
1.1	Graph Model & Problem Setting	3
2	The Core Idea (Layers + Queue)	5
2.1	The core idea of BFS	5
2.2	Distance layers	5
2.3	Why layers matter	5
2.4	The queue discipline	5
2.5	Wavefront interpretation	5
2.6	Engineering interpretation	5
2.7	Conceptual summary	6
3	The Algorithm (CLRS Pseudocode)	7
3.1	Roadmap	7
3.2	Objective	7
3.3	Formal model	7
3.4	Vertex state	7
3.5	CLRS pseudocode — BFS	7
3.6	Structural guarantees	8
3.7	Why π produces a tree	8
3.8	Time complexity	8
4	Correctness Proof	9
4.1	Roadmap	9
4.2	What must be proven	9
4.3	Key invariant	9
4.4	Lemma 1 — distances never underestimate	9
4.5	Lemma 2 — FIFO enforces layered exploration	9
4.6	Lemma 3 — first discovery is optimal	10
4.7	Theorem — BFS computes shortest paths (unweighted)	10
4.8	The BFS tree is a shortest-path tree	10
4.9	Time complexity (revisited)	10
4.10	Structural insight	10

5	Complexity	11
5.1	Roadmap	11
5.2	Time complexity (adjacency list)	11
5.3	Vertex processing	11
5.4	Edge examination	11
5.5	Adjacency matrix case	11
5.6	Space complexity	11
5.7	Structural insight	11
6	BFS Tree & Path Reconstruction	12
6.1	Roadmap	12
6.2	What BFS produces	12
6.3	The predecessor subgraph	12
6.4	Why this is a tree	12
6.5	Small formal diagram (layers + tree)	12
6.6	Path reconstruction (definition)	13
6.7	CLRS-style procedure — PRINT-PATH	13
6.8	Iterative reconstruction (engineering-friendly)	13
6.9	Worked example	14
6.10	Engineering patterns	14
7	Applications	15
7.1	Roadmap	15
7.2	Shortest paths (unweighted)	15
7.3	Grid pathfinding (games & robotics)	15
7.4	Reachability & connectivity	15
7.5	Connected components (undirected)	15
7.6	Bipartite checking (two-coloring)	16
7.7	Levels, broadcast, degrees of separation	16
7.8	Engineering patterns	16
8	BFS vs DFS	17
8.1	Roadmap	17
8.2	Core difference: exploration order	17
8.3	Queue vs stack	17
8.4	Side-by-side comparison	17
8.5	What BFS produces	18
8.6	What DFS produces	18
8.7	When to choose BFS	18
8.8	When to choose DFS	18
8.9	Complexity and practical memory	18
8.10	Pitfalls & guidelines	18
	Source Pages	19

1 Graph Model & Problem Setting

1.1 Graph Model & Problem Setting

Note

Before we write any code, we must be precise about the mathematical object and the exact problem. BFS is not merely a “visit neighbors” routine: it is a shortest-path algorithm for a specific setting.

1.1.1 Why start with the model?

We begin with the graph:

$$G = (V, E)$$

where:

- V is the set of vertices;
- $E \subseteq V \times V$ is the set of edges.

1.1.2 Directed and undirected graphs

Definition

Directed graph (digraph). Edges are ordered pairs: $E \subseteq V \times V$. If $(u, v) \in E$, you can go from u to v (not necessarily the other way).

Definition

Undirected graph. Edges are unordered: $E \subseteq \{\{u, v\} \mid u, v \in V\}$. Adjacency is symmetric.

1.1.3 Graph representation

Note

Algorithms operate on a representation of G .

Definition

Adjacency list. For each vertex u , store:

$$Adj[u] = \{v \in V \mid (u, v) \in E\}.$$

This is the standard in CLRS and yields $O(|V| + |E|)$ traversal time.

Definition

Adjacency matrix. A matrix $A \in \{0, 1\}^{|V| \times |V|}$ with:

$$A[u, v] = \begin{cases} 1, & \text{if } (u, v) \in E \\ 0, & \text{otherwise} \end{cases}$$

This often leads to $O(|V|^2)$ neighbor scanning.

1.1.4 The problem BFS solves

Let $s \in V$ be the source.

Definition
Reachability.

$$\text{Reach}(s) = \{v \in V \mid \exists \text{ a path from } s \text{ to } v\}.$$

Definition

Unweighted shortest-path distance. $\delta(s, v)$ is the minimum number of edges among all paths from s to v .

1.1.5 Paths (formal)

A path of length k is a sequence $v_0, v_1, \dots, v_k$ such that $(v_{i-1}, v_i) \in E$ for all $i = 1, \dots, k$. Its length is k .

1.1.6 Layer structure (preview)

Define distance layers:

$$L_i = \{v \in V \mid \delta(s, v) = i\}.$$

So $L_0 = \{s\}$, L_1 are neighbors of s , L_2 are vertices at distance 2, etc. BFS will explore exactly in this layered order.

1.1.7 BFS objective (precise)

Definition

Given $G = (V, E)$ and a source $s \in V$, compute:

- a distance function $d : V \rightarrow \mathbb{N} \cup \{\infty\}$, with $d[v] = \delta(s, v)$ for reachable vertices;
- a predecessor function $\pi : V \rightarrow V \cup \{\text{NIL}\}$ such that the π pointers form a shortest-path tree rooted at s .

1.1.8 Why this matters (engineering)

Typical appearances: hop-count routing, social distance, crawling, dependency analysis, connectivity, bipartite testing. In discrete-math terms, BFS builds the metric structure induced by edge-count distance.

2 The Core Idea (Layers + Queue)

2.1 The core idea of BFS

Note

BFS has a geometry: it expands from the source like a wave. The algorithm is best understood as exploration *by distance layers*.

2.2 Distance layers

Let $G = (V, E)$ and $s \in V$. Recall $\delta(s, v)$ and:

$$L_i = \{ v \in V \mid \delta(s, v) = i \}.$$

These layers partition reachable vertices: $L_0 = \{s\}$, L_1 neighbors of s , L_2 vertices at distance 2, and so on.

2.3 Why layers matter

Key guarantee: if a vertex is discovered in layer L_i , then no shorter path exists. This only holds because BFS finishes all of L_i before moving to L_{i+1} .

2.4 The queue discipline

Definition

BFS uses a queue (FIFO):

- vertices are enqueued at discovery time;
- vertices are dequeued in the same order they were inserted.

This FIFO rule forces the layer order.

2.5 Wavefront interpretation

Think in steps i :

- process all vertices in L_i ;
- newly discovered neighbors become L_{i+1} .

The boundary between explored and unexplored is the *wavefront*.

2.6 Engineering interpretation

Why the guarantees hold:

- edges have equal weight (unweighted);
- distance is measured by edge count;
- FIFO ensures “shortest distance first”.

Hence, for every reachable v , BFS yields $d[v] = \delta(s, v)$.

2.7 Conceptual summary

BFS = Layered geometry + FIFO discipline + Wavefront expansion.

3 The Algorithm (CLRS Pseudocode)

3.1 Roadmap

In this part we:

- state the objective precisely;
- define the formal model and vertex state;
- present the classic CLRS pseudocode;
- explain why π pointers form a tree;
- analyze time complexity.

3.2 Objective

Given a graph G and source s , BFS explores vertices in increasing distance from s ; in unweighted graphs it computes shortest-path distances (in number of edges) to every reachable vertex.

3.3 Formal model

Let $G = (V, E)$ and $s \in V$. Assume adjacency lists $Adj[u]$ to iterate neighbors efficiently.

3.4 Vertex state

Each vertex $v \in V$ stores:

- `color[v] ∈ {WHITE, GRAY, BLACK}`;
- `d[v]` (distance from s);
- `π[v]` (predecessor).

Definition

Color meaning:

- WHITE: not discovered;
- GRAY: discovered, not fully explored;
- BLACK: fully explored.

3.5 CLRS pseudocode — BFS

Example

`BFS(G, s):`

```
for each vertex u in V[G]:  
    color[u] = WHITE  
    d[u] =  
    [u] = NIL
```

```
color[s] = GRAY
d[s] = 0
[s] = NIL

Q = empty queue
ENQUEUE(Q, s)

while Q is not empty:
    u = DEQUEUE(Q)
    for each v in Adj[u]:
        if color[v] == WHITE:
            color[v] = GRAY
            d[v] = d[u] + 1
            [v] = u
            ENQUEUE(Q, v)

    color[u] = BLACK
```

3.6 Structural guarantees

BFS guarantees:

- exploration in non-decreasing distance order;
- first discovery gives a shortest path (unweighted);
- predecessor pointers $\pi[v]$ form a BFS tree.

3.7 Why π produces a tree

Every reachable vertex (except s) receives exactly one predecessor, so the predecessor relation defines a tree structure over the reachable vertices.

3.8 Time complexity

With adjacency lists:

$$O(|V| + |E|).$$

Each vertex enters the queue at most once; each edge is examined at most twice in the undirected case.

4 Correctness Proof

4.1 Roadmap

We:

- state the correctness claim;
- define the core invariant;
- show BFS never underestimates distances;
- use FIFO to justify layers;
- prove first discovery is optimal;
- conclude $d[v] = \delta(s, v)$ for reachable vertices;
- show π forms a shortest-path tree;
- revisit complexity and extract the key structural insight.

4.2 What must be proven

Let $G = (V, E)$ and $s \in V$. For each vertex v , let $\delta(s, v)$ be the true shortest-path distance (edge count). We must show that BFS computes:

$$d[v] = \delta(s, v)$$

for every vertex reachable from s .

4.3 Key invariant

Definition

Invariant. When a vertex v is first discovered (WHITE → GRAY), BFS assigns $d[v] = \delta(s, v)$.

4.4 Lemma 1 — distances never underestimate

Note

BFS sets $d[v]$ only via an edge $(u, v) \in E$ as $d[v] = d[u] + 1$. This cannot be smaller than the true optimum, so BFS never assigns a value below $\delta(s, v)$.

4.5 Lemma 2 — FIFO enforces layered exploration

Definition

Layer property. All vertices with distance k are dequeued before any vertex with distance $k + 1$.

Reason: when u is dequeued, newly discovered neighbors are assigned $d[u] + 1$ and appended to the end of the queue, creating consecutive layers.

4.6 Lemma 3 — first discovery is optimal

Proof idea (contradiction): assume BFS discovers v with $d[v] > \delta(s, v)$. Take a shortest path

$$s = v_0, v_1, \dots, v_k = v.$$

Then v_{k-1} is in the previous layer, is dequeued earlier, and would discover v with distance $\delta(s, v)$ —a contradiction. Hence first discovery assigns the true shortest distance.

4.7 Theorem — BFS computes shortest paths (unweighted)

From Lemma 3, each reachable vertex satisfies $d[v] = \delta(s, v)$. Therefore BFS is correct for unweighted shortest-path distances.

4.8 The BFS tree is a shortest-path tree

When $\pi[v]$ is set to u , BFS enforces $d[v] = d[u] + 1$. Following predecessors strictly decreases distance, yielding a path from v back to s of length $d[v]$, which equals $\delta(s, v)$.

4.9 Time complexity (revisited)

The correctness proof does not change runtime:

$$O(|V| + |E|).$$

4.10 Structural insight

Takeaway: the queue is not an implementation detail. FIFO is the structural reason that the layered invariant (and the proof) works.

5 Complexity

5.1 Roadmap

We analyze:

- time with adjacency lists;
- why each vertex is processed once;
- why each edge is examined at most twice;
- the adjacency matrix case;
- space complexity;
- the structural takeaway.

5.2 Time complexity (adjacency list)

$$O(|V| + |E|).$$

This is optimal for traversal.

5.3 Vertex processing

Each vertex is:

- discovered once;
- enqueued once;
- dequeued once;
- colored BLACK once.

So the vertex contribution is $O(|V|)$.

5.4 Edge examination

In undirected graphs, each edge can appear in adjacency lists twice, so scanning edges contributes $O(|E|)$.

5.5 Adjacency matrix case

If neighbors are checked by scanning a row, cost becomes:

$$O(|V|^2).$$

Hence, for sparse graphs, adjacency lists are asymptotically superior.

5.6 Space complexity

$$O(|V|)$$

for the queue and arrays `color`, `d`, and `$\pi$` .

5.7 Structural insight

No revisits, no redundant exploration, and strict layer order—enforced by FIFO.

6 BFS Tree & Path Reconstruction

6.1 Roadmap

We:

- make explicit what BFS outputs (d and π);
- define the predecessor subgraph and show it is a tree;
- formalize path reconstruction from π pointers;
- give CLRS-style recursive printing and an iterative version;
- work a small example;
- highlight engineering patterns (routing, breadcrumbs, impact analysis).

6.2 What BFS produces

After running BFS from s , each vertex v has:

- $d[v]$: shortest distance from s to v (in edges);
- $\pi[v]$: predecessor of v on a shortest path (or NIL).

Key rule at discovery: if $\pi[v] = u$, then $d[v] = d[u] + 1$.

6.3 The predecessor subgraph

Define:

$$E_\pi = \{ (\pi[v], v) \mid v \in V, v \neq s, \pi[v] \neq NIL \}.$$

The graph $T = (V, E_\pi)$ is the BFS predecessor subgraph. Restricted to reachable vertices, it forms a tree rooted at s .

6.4 Why this is a tree

Two properties imply a tree:

- **Unique parent.** Each reachable $v \neq s$ gets exactly one predecessor.
- **Acyclic.** Following predecessors strictly decreases distance: $d[\pi[v]] = d[v] - 1$.

So from any reachable v , repeatedly following π must reach s and the path is unique.

6.5 Small formal diagram (layers + tree)

Example

Layer + tree view:

```

L0:   s
      |
L1:  a  b
      |  |

```

```

L2:  c    d    e
      |
L3:      f

```

Edges are $([v], v)$. Each edge goes from layer k to layer $k+1$.

6.6 Path reconstruction (definition)

A reconstructed path to a reachable v is:

$$s = v_0, v_1, \dots, v_k = v \quad \text{where} \quad v_{i-1} = \pi[v_i].$$

Its length is $k = d[v]$.

6.7 CLRS-style procedure — PRINT-PATH

Example

```

PRINT-PATH(, s, v):
    if v == s:
        print s
    else if [v] == NIL:
        print "no path"
    else:
        PRINT-PATH(, s, [v])
        print v

```

6.8 Iterative reconstruction (engineering-friendly)

Example

```

RECONSTRUCT-PATH(, s, v):
    path = []
    cur = v

    while cur != NIL:
        path.append(cur)
        if cur == s:
            break
        cur = [cur]

    if path.last != s:
        return []    // no path

    reverse(path)

```

return path

6.9 Worked example

Suppose BFS produced:

v	$d[v]$	$\pi[v]$
s	0	NIL
a	1	s
b	1	s
d	2	b
f	3	d

To reconstruct f : follow predecessors $f \rightarrow d \rightarrow b \rightarrow s$, then reverse to get $s \rightarrow b \rightarrow d \rightarrow f$.

6.10 Engineering patterns

BFS trees and parent pointers appear in routing (hop-by-hop), breadcrumbs (UI navigation), hierarchies, and dependency impact (“what depends on this?”). The pointer array π is the concrete structure that turns BFS into a usable output.

7 Applications

7.1 Roadmap

We connect BFS to:

- shortest paths in unweighted graphs;
- grids (games, robotics);
- reachability / connectivity;
- connected components (undirected);
- bipartite checking (two-coloring);
- levels, broadcasting, degrees of separation;
- engineering patterns (routing, breadcrumbs, dependency impact).

7.2 Shortest paths (unweighted)

When each edge has unit cost, BFS computes:

$$d[v] = \delta(s, v),$$

and π allows reconstructing an actual shortest path.

7.3 Grid pathfinding (games & robotics)

Model a grid as a graph:

- each cell is a vertex;
- edges represent allowed moves (4-neighborhood or 8-neighborhood);
- obstacles remove vertices or edges.

BFS finds the minimum number of moves from start to goal.

7.4 Reachability & connectivity

BFS answers “can I get there from here?”: run BFS from s ; all discovered vertices are reachable. Useful in access graphs, dependency reachability, and network connectivity checks.

7.5 Connected components (undirected)

To find all components:

1. mark all vertices unvisited;
2. for each unvisited vertex, run BFS and mark the entire discovered set as one component;
3. each BFS run discovers exactly one component.

Total time remains $O(|V| + |E|)$.

7.6 Bipartite checking (two-coloring)

Two-color layers:

- assign color A to s ;
- neighbors must be color B;
- neighbors of B must be A, etc.;
- if any edge connects same-color vertices, the graph is not bipartite.

7.7 Levels, broadcast, degrees of separation

BFS naturally computes levels:

$$L_k = \{ v \in V \mid d[v] = k \}.$$

This models hop-based broadcast, discrete-time diffusion, and “friends within k hops” queries.

7.8 Engineering patterns

BFS is a reusable pattern: routing by hop-count, breadcrumbs/parents, impact analysis in dependency graphs, and state-space search with unit transitions. If your system is “states + unit transitions”, BFS is typically the first tool to try.

8 BFS vs DFS

8.1 Roadmap

We compare:

- exploration order;
- queue vs stack (or recursion);
- typical outputs / metadata;
- when BFS is the right tool;
- when DFS is the right tool;
- complexity and memory profile;
- pitfalls and engineering guidelines.

8.2 Core difference: exploration order

Definition

BFS explores by increasing distance from the source (layer by layer). DFS explores by going as deep as possible, then backtracks.

The order changes guarantees: BFS yields shortest paths in unweighted graphs; DFS does not.

8.3 Queue vs stack

Note

The data structure dictates behavior:

- BFS: queue (FIFO) → preserves layers;
- DFS: stack (LIFO) explicit, or recursion (implicit call stack).

Rule of thumb: FIFO “expands equally”; LIFO “commits to a path”.

8.4 Side-by-side comparison

Aspect	BFS	DFS
Exploration	Layer by layer	Deep then backtrack
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Shortest paths (unweighted)	Yes	No
Typical outputs	$d + \pi$ tree	discovery/finish times, DFS forest
Best for	min hops, layers	structure, cycles, topo order, SCC
Complexity	$O(V + E)$	$O(V + E)$

8.5 What BFS produces

Two primary artifacts:

- $d[v]$: shortest distance from s to v (edges);
- $\pi[v]$: predecessor pointers forming a BFS tree.

Key guarantee: $d[v] = \delta(s, v)$ in unweighted graphs.

8.6 What DFS produces

DFS is a “structure extractor”. Common metadata: discovery time $tin[v]$, finish time $tout[v]$, DFS forest, and (in digraphs) edge classification.

8.7 When to choose BFS

Use BFS when transitions are unit-cost and you care about: minimum steps, nearest target (early exit), layers/wavefront, and grid shortest moves. One-liner: *If each move costs 1, try BFS first.*

8.8 When to choose DFS

Use DFS for deeper global properties: topological sorting, cycle detection (esp. directed), SCC (Kosaraju/Tarjan), and backtracking/state-space exploration. One-liner: *If you need ordering/structure, use DFS.*

8.9 Complexity and practical memory

With adjacency lists, both are $O(|V| + |E|)$. Memory differs: BFS can hold a wide frontier (large queue), DFS holds a deep path (depth-dependent). Recursive DFS can overflow on very deep graphs.

8.10 Pitfalls & guidelines

- Don’t use DFS for shortest paths (wrong tool).
- In BFS, mark visited at *enqueue* time (avoids duplicates).
- For disconnected graphs, run BFS/DFS from each unvisited vertex to cover everything.
- BFS is not for weighted shortest paths → use Dijkstra / 0-1 BFS.

Guideline: start with the question: *Do I need the nearest solution, or do I need structure?*

Source Pages

- [Part 1 — Graph Model & Problem Setting](#)
- [Part 2 — The Core Idea \(Layers + Queue\)](#)
- [Part 3 — The Algorithm \(CLRS Pseudocode\)](#)
- [Part 4 — Correctness Proof](#)
- [Part 5 — Complexity](#)
- [Part 6 — BFS Tree & Path Reconstruction](#)
- [Part 7 — Applications](#)
- [Part 8 — BFS vs DFS](#)